LAPORAN PERTEMUAN 1-7
APLIKASI KONSOL PERPUSTAKAAN
PRAKTIKUM PEMROGRAMAN BERBASIS OBJEK

Disusun oleh:

Nama : Elsa Marthalinda

NIM : 254107020204

Kelas : 2B

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

OKTOBER 2026

Bagian 1 – Pertemuan 1: Pengantar

Class Main.java;

```java
package id.ac.polinema.library;
import java.util.Scanner;
public class Main {

    Run | Debug
    public static void main(String[] args) {
        System.out.println("Welcome to Polinema Library");
    }
}
```

Gambar 1: Class Main.java awal

Program ini berada dalam paket id.ac.polinema.library dan mendefinisikan kelas Main yang namanya disesuaikan dengan nama berkas. Eksekusi dimulai dari method main yang dipanggil langsung oleh JVM untuk menjalankan perintah System.out.println, yaitu mencetak teks ke layar lalu berpindah baris, sedangkan bagian komentar di dalamnya diabaikan saat program berjalan.

Pikirkan Dulu:

1. Apa yang terjadi saat kamu menekan Run: apa yang dikerjakan compiler, dan apa yang dikerjakan JVM?
   Jawab: Compiler bertugas memeriksa sintaks dan mengubah kode menjadi bytecode (file .class) sebelum program berjalan. Jika bebas error, JVM akan memuat dan menerjemahkan bytecode tersebut ke instruksi mesin saat program dijalankan.
2. Method mana yang pertama kali dijalankan Java, dan mengapa harus static?
   Jawab: Method public static void main(String[] args). Saat program mulai, belum ada satu pun objek di memori. Karena method static milik kelas (bukan milik objek), JVM bisa memanggilnya langsung dengan Main.main(...) tanpa perlu membuat new Main() terlebih dahulu. Kalau main tidak static, JVM tidak punya objek untuk memanggilnya.

Uji Pemahaman:

1. Sebutkan 5 benda di perpustakaan yang menurutmu adalah *objek*. Untuk masing-masing, tulis satu *data* yang dimilikinya dan satu *aksi* yang bisa dilakukannya.
   Jawab:

Objek	Satu data (atribut)	Satu aksi (method)
Buku “Clean Code”	Judul = Clean Code, tahun = 2008	Dipinjam (checkOut)
Angota	IDE anggota = M001	Meminajm koleksi
Kartu perpustakan	Nomor kartu = CARD-M001	Menunjukkan nomor kartu (getNumber)

Rak buku	Kode rak, kapasitas 5 buku	Menerima buku (addBook)
Petugas perpustakaan	Nama petugas	Meminjam buku ke anggota (lend)

Bagian 2 – Pertemuan 2: Class dan Object

Class Author.java; dan Class Book.java;

```java
public class Author {
    private String name;
    private String country;
    /** Membuat penulis dengan nama dan negara asal. */
    public Author(String name, String country) {
        this.name = name;
        this.country = country;
    }

    public String getName() {
        return name;
    }

    public String getCountry() {
        return country;
    }

    /** Teks berbentuk "nama (negara)". Contoh: Andrea Hirata (Indonesia). */
    public String getInfo() {
        return name + " (" + country + ")";
    }
}
```

Gambar 2: Pembuatan class Author.java;

Hubungan antara kelas Book dan Author ini menerapkan konsep **asosiasi** (khususnya *agregasi*), yaitu kondisi ketika satu kelas menggunakan kelas lain sebagai tipe data untuk atributnya. Konsep ini dipakai agar struktur data program rapi dan terpisah sesuai fungsinya di dunia nyata, alih-alih menumpuk semua data penulis ke dalam kelas buku. Konsep ini digunakan di dalam kelas Book, di mana objek Author disimpan sebagai atribut untuk mengelola informasi pembuat buku. Hasilnya, kelas Book bisa mengambil detail data seperti nama atau negara asal penulis dengan memanggil method dari kelas Author tersebut.

Pikirkan Dulu:

1. Data apa saja yang perlu diingat setiap objek Book? Mana yang berasal dari parameter, mana yang diatur sendiri oleh objek?
   Jawab: Data yang perlu diingat adalah isbn, title, year, available, dan author. Data yang berasal dari parameter constructor adalah isbn, title, year (dan author jika menggunakan constructor kedua). Sedangkan data yang diatur sendiri oleh objek saat pertama kali dibuat adalah status ketersediaan available (di-set nilai awalnya true).
2. Agar objek tahu sedang dipinjam atau tidak, apa yang harus diingatnya?
   Jawab: Objek harus mengingat atribut/field tipe boolean available. Jika nilainya true, artinya buku sedang tersedia (tidak dipinjam), sedangkan jika false, artinya buku sedang dipinjam.

3. Di constructor, nama parameter sama dengan nama field. Bagaimana Java membedakannya?
   Jawab: Java membedakannya dengan menggunakan kata kunci this. Variabel yang diawali this. (seperti this.isbn) merujuk pada field/atribut milik objek kelas tersebut, sedangkan variabel tanpa this (seperti isbn) merujuk pada parameter yang diterima oleh constructor.

Uji Pemahaman:

1. Perhatikan kode ini, lalu tebak outputnya.

```
Book a = new Book("1", "Clean Code", 2008);
Book b = a;
b.checkOut();
System.out.println(a.isAvailable());
Book c = new Book("2", "Refactoring", 1999);
System.out.println(c.isAvailable());
```

   Mengapa hasil baris pertama dan kedua berbeda? Gambar a, b, dan c di stack, dan objeknya di heap.
   Jawab: Baris pertama **false**, baris kedua **true**. Variabel a dan b sama-sama menyimpan referensi (alamat) ke satu objek Book yang sama di heap, karena Book b = a hanya menyalin referensi, bukan objeknya. Jadi b.checkOut() mengubah objek yang juga dilihat oleh a, sehingga a.isAvailable() menjadi false. Variabel c dibuat dengan new sehingga menunjuk objek lain yang terpisah, dan statusnya masih true.

Bagian 3 – Pertemuan 3: Enkapsulasi

Class Book.java;, Member.java;, dan Author.java;

```
public class Book extends LibraryItem {
    private String isbn;
    private String title;
    private int year;
    private boolean available;

    private Author author;
```

Gambar 3: Pengubahan field menjadi private pada class Book.java;

Konsep utama yang diterapkan pada kode tersebut adalah **Enkapsulasi** dalam Pemrograman Berbasis Objek. Dengan enkapsulasi teknik menyembunyikan data internal suatu objek dengan mengubah modifier atribut menjadi private dan membatasi aksesnya hanya melalui method seperti *getter* dan *setter*. Konsep ini dipakai untuk menjaga integritas data serta mencegah perubahan nilai secara langsung dari luar kelas yang tidak valid, seperti memasukkan judul kosong atau batas pinjaman yang tidak sesuai. Penerapan enkapsulasi ini dilakukan pada kelas **Book**, **Member**, dan **Author** dengan cara mengamankan semua data atributnya serta menambahkan aturan validasi pada method terkait.

Pikirkan Dulu:

1. Jika Member punya public int loanCount, baris kode apa yang merusak aturan "maksimal 3"? Bagaimana private mencegahnya?
   Jawab: member.loanCount = 99; (akses langsung dari luar kelas). Pencegahannya dengan Keyword private melarang akses/perubahan variabel secara langsung dari luar kelas, sehingga perubahan wajib melalui method addLoan() yang dibatasi aturan loanCount < 3.
2. Constructor dan setter punya aturan validasi yang sama. Bagaimana menghindari menulis aturannya dua kali?
   Jawab: Memanggil setter di dalam constructor (contoh: setName(name);). Dengan begitu, aturan validasi cukup ditulis sekali di dalam method setter.
3. Apakah canBorrow() perlu field sendiri? Apa yang terjadi jika datanya tidak sinkron dengan loanCount?
   Jawab: Tidak perlu field sendiri. Nilainya cukup dihitung langsung (*dynamic return*) dari loanCount < 3. Jika dibuat sebagai variabel tersendiri (misal boolean canBorrow), variabel tersebut berisiko lupa diperbarui saat loanCount berubah, sehingga status kelayakan pinjam menjadi tidak akurat/inkonsisten.
4. Mengapa memberId tidak boleh punya setter?
   Jawab: Karena memberId merupakan identitas unik yang bersifat permanen (*read-only*). Jika mengubah ID di tengah jalan dapat merusak integritas data dan riwayat transaksi anggota.

Bagian 4 – Pertemuan 4: Relasi Class

Class Shelft.java;, Librarian.java;, LibraryCard.java;, Book.java;, Member.java;, Author.java;,

```java
public Member(String memberId, String name) {
    if (memberId == null || memberId.trim().isEmpty()) {
        throw new IllegalArgumentException("Member ID cannot be empty");
    }
    this.memberId = memberId;
    setName(name);
    this.loanCount = 0;
    this.card = new LibraryCard("CARD-" + memberId);
}
```

Gambar 4: Class Member.java;

```java
public class Shelf {
    private String code;
    private Book[] books;
    private int count;

    public boolean addBook(Book book) {
        if (book == null || count >= books.length) {
            return false;
        }
        books[count] = book;
        count++;
        return true;
    }
```

Gambar 5: Class Shelf.java;

```java
public class Librarian {
    public boolean lend(Member member, Book book) {
        if (member == null || book == null) {
            return false;
        }
        if (member.canBorrow() && book.isAvailable()) {
            if (book.checkOut()) {
                member.addLoan();
                return true;
            }
        }
        return false;
    }
```

Gambar 6: Class Librarian.java;

Objek jarang berdiri sendiri, jadi kelas punya relasi. Asosiasi adalah hubungan longgar (Book mengenal Author). Agregasi adalah hubungan "memiliki" yang lemah: bagian bisa hidup tanpa pemiliknya (Shelf berisi Book). Komposisi adalah hubungan "memiliki" yang kuat: bagian dibuat dan hilang bersama pemiliknya (Member dan LibraryCard). Dependensi berarti kelas hanya memakai kelas lain lewat parameter method tanpa menyimpannya (Librarian memakai Member dan Book). Membedakan keempatnya penting.

Pikirkan Dulu:

1. Siapa yang membuat objek di ujung garis: kelas ini sendiri, atau kode di luarnya?
   Jawab: Pada relasi komposisi (Member ke LibraryCard), objek dibuat langsung oleh kelas Member itu sendiri di dalam constructor-nya. Sedangkan pada relasi agregasi (Shelf ke Book), asosiasi (Book ke Author), dan dependensi (Librarian ke Member/Book), objek dibuat oleh kode di luar kelas tersebut dan disalurkan melalui constructor, method setter, atau parameter method.
2. Jika pemilik objek dibuang, apakah objek di ujung garis ikut hilang?
   Jawab: Jika pemilik (Member) dibuang/dihapus, maka objek LibraryCard di ujung garis ikut hilang. Namun pada relasi agregasi (Shelf ke Book) dan asosiasi (Book ke Author), objek di ujung garis tidak ikut hilang dan tetap eksis secara independen di memori.
3. Apakah objek di ujung garis perlu disimpan sebagai field, atau cukup muncul sebagai parameter method?
   Jawab: Wajib disimpan sebagai field jika memiliki relasi komposisi (card pada Member), agregasi (books pada Shelf), atau asosiasi (author pada Book). Sebaliknya, pada relasi dependensi (Librarian), objek tidak perlu disimpan sebagai field melainkan cukup muncul sementara sebagai parameter method (lend dan receive).
4. Apa arti 0..5 dan 0..1 pada garis tersebut?
   Jawab: Angka 0..5 pada garis agregasi berarti sebuah rak (Shelf) dapat menampung minimal 0 hingga maksimal 5 buah buku (Book). Angka 0..1 pada garis asosiasi berarti sebuah buku (Book) dapat terhubung dengan 0 penulis (belum diset) atau paling banyak 1 penulis (Author).

Uji Pemahaman:

1. Jika objek Member dibuang, apa yang terjadi pada LibraryCard-nya? Jika objek Shelf dibuang, apa yang terjadi pada Book di dalamnya? Jelaskan perbedaan keduanya dengan kata-katamu sendiri.
   Jawab: Jika objek Member dibuang, LibraryCard-nya ikut hilang. Kartu dibuat oleh Member sendiri dan hanya dimiliki oleh Member itu, sehingga tidak ada lagi yang memegangnya (komposisi: hidup dan matinya terikat). Jika objek Shelf dibuang, Book di dalamnya tidak ikut hilang. Buku dibuat di luar rak dan masih bisa dipegang oleh bagian lain program, misalnya Library atau variabel di Main; rak hanya "menampung" referensinya (agregasi). Perbedaannya: pada komposisi bagian tidak punya hidup sendiri, pada agregasi bagian punya hidup sendiri.

Bagian 5 – Pertemuan 6: Inheritance

Class LibraryItem(Parent Class), Dvd(Subclass), Magazine(Subclass)

public class Book extends LibraryItem
public class Dvd extends LibraryItem
public class Magazine extends LibraryItem

Tabel 1: Class pewarisanLibraryItem pada Book, Dvd dan Magazine

Kode di atas menerapkan konsep **Inheritance** (Pewarisan) dalam Pemrograman Berbasis Objek. Konsep ini memungkinkan suatu kelas turunan (*subclass*) untuk mewarisi atribut umum dan method milik kelas induk (*superclass*). Pewarisan dipakai untuk mencegah pengulangan kode (*code duplication*) serta memudahkan pengelolaan logika umum—seperti judul, tahun, dan ketersediaan barang—di satu tempat saja. Dalam kode tersebut, saya memakainya pada kelas **Book**, **Dvd**, dan **Magazine** yang menjadi *subclass* langsung dan mewarisi properti dasar dari kelas induknya, yaitu **LibraryItem**.

Pikirkan Dulu:

1. Bandingkan Book di diagram Bagian 4 dengan Dvd di diagram Bagian 5. Apa yang sama di keduanya? Apa yang khusus?
   Jawab: Persamaannya adalah keduanya mewarisi sifat umum dari LibraryItem (memiliki judul, tahun, ketersediaan, serta method *getter/setter/checkOut/returnItem*). Perbedaannya, Book memiliki atribut khusus isbn dan author, sedangkan Dvd memiliki atribut khusus durationMinutes.
2. Kalimat "Dvd adalah sebuah LibraryItem" masuk akal. Apakah "Library adalah sebuah Book" juga masuk akal?
   Jawab: Tidak masuk akal. Kalimat "Dvd adalah sebuah LibraryItem" masuk akal karena Dvd merupakan salah satu jenis barang perpustakaan (relasi IS-A). Perpustakaan (*Library*) adalah tempat/wadah yang menampung barang, bukan jenis dari barang itu sendiri.

3. Mengapa title dan year ditandai # dan bukan - ?
   Jawab: Karena simbol # dalam diagram UML menandakan bahwa atribut tersebut memiliki akses kontrol protected, yang artinya hanya bisa diakses oleh kelas LibraryItem itu sendiri dan kelas-kelas turunan (*subclass*) yang mewarisinya.
4. Jika LibraryItem punya constructor yang meminta title dan year, siapa yang harus mengisinya saat sebuah Dvd dibuat?
   Jawab: Konstruktor Dvd itu sendiri yang bertanggung jawab meneruskan nilai title dan year ke parent dengan memanggil perintah super(title, year) di baris pertama constructor-nya.

Uji Pemahaman:

1. Apa yang diwarisi Dvd dari LibraryItem, dan apa yang tidak? Mengapa constructor LibraryItem tidak ikut diwariskan, tetapi tetap harus dipanggil saat Dvd dibuat?
   Jawab: Pada kelas Dvd, hal yang diwarisi dari LibraryItem meliputi atribut title, year, dan available, serta seluruh method publiknya seperti *getter*, *setter*, checkOut(), dan returnItem(). Sebaliknya, *constructor* milik LibraryItem tidak ikut diwariskan karena bersifat khusus untuk menginisialisasi kelasnya sendiri. Meskipun begitu, *constructor parent* tersebut tetap wajib dipanggil menggunakan perintah super() agar seluruh atribut awal milik LibraryItem dapat diinisialisasi dan divalidasi dengan benar ketika objek Dvd dibuat.

Bagian 6 – Pertemuan 7: Overriding dan Overloading

Class Book extends LibraryItem, Dvd extends LibraryItem, Magazine extends LibraryItem

```
public void extendLoan() {
    extendLoan(days: 7);
}

public void extendLoan(int days) {
    if (days <= 0) {
        throw new IllegalArgumentException("Days must be greater than 0");
    }
    this.extendedDays += days;
}
```

Gambar 7: Implementation Method Overloading pada Kelas LibraryItem

```
public Book(String isbn, String title, int year, Author author) {
    this(isbn, title, year);
    this.author = author;
}

@Override
public int loanDays() {
    return 14;
}

@Override
public String describe() {
    String authorName = (author != null && author.getName() != null) ? author.getName() : "Unknown";
    return super.describe() + " by " + authorName;
}
```

Gambar 8: Implementation Constructor Overloading dan Method Overriding pada Kelas Book

Overriding terjadi ketika subclass menulis ulang method milik induk dengan signature yang sama agar perilakunya sesuai jenisnya, ditandai @Override. Contohnya loanDays(): LibraryItem mengembalikan 7, Book menjadi 14, Dvd menjadi 3. Overloading adalah beberapa method atau constructor dengan nama yang sama tetapi daftar parameter berbeda, misalnya extendLoan() dan extendLoan(int), atau dua constructor Book dan Magazine. Keduanya dipakai agar kode fleksibel tanpa duplikasi, dan hasilnya dipakai di Book, Dvd, Magazine, dan LibraryItem.

Pikirkan Dulu:

1. Bandingkan describe() milik Book dan milik LibraryItem. Bagian mana yang sudah dikerjakan parent? Perlukah kamu menulisnya ulang?
   Jawab: Parent sudah menghasilkan bagian judul (tahun). Book tidak perlu menulis ulang bagian itu; cukup memanggil super.describe() lalu menambahkan " by " + penulis.
2. Magazine tidak perlu loanDays() sendiri. Mengapa? Versi siapa yang berjalan?
   Jawab: Lama pinjam majalah 7 hari, sama dengan nilai default di LibraryItem, jadi Magazine cukup mewarisinya. Saat loanDays() dipanggil pada objek Magazine, yang berjalan adalah versi milik LibraryItem.
3. extendLoan() dan extendLoan(int) sama-sama bernama extendLoan. Apa yang membedakannya bagi compiler? Bisakah salah satu memakai yang lain?
   Jawab: Compiler membedakannya dari daftar parameter (jumlah dan tipe), yaitu signature-nya. Tipe kembalian saja tidak cukup untuk membedakan. Bisa: extendLoan() memanggil extendLoan(7), sehingga logika penambahan hanya ada di satu tempat.
4. Dua constructor Book hampir sama. Bagaimana menghindari menyalin isinya?
   Jawab: Gunakan kata kunci this(...) pada constructor 4-parameter untuk memanggil constructor 3-parameter (this(isbn, title, year);), sehingga proses validasi dan inisialisasi judul, tahun, serta ISBN ditangani di satu tempat saja.

Bagian 7 – Gabungan Pertemuan 1-7: Aplikasi Konsol

Class LibraryApp,java;, class Library.java;, dan class Main.java;

```
Library library = new Library(name: "Polinema Library");
Author author = new Author(name: "Robert Martin", country: "USA");
Book book = new Book(isbn: "978-0132350884", title: "Clean Code", year: 2008, author);
Dvd dvd = new Dvd(title: "Inception", year: 2010, duration: 148);
Magazine magazine = new Magazine(title: "Tempo", year: 2024, issueNumber: 7);
Member member = new Member(memberId: "M001", name: "Budi");

library.addItem(book);
library.addItem(dvd);
library.addItem(magazine);
library.addMember(member);

LibraryApp app= new LibraryApp(library, new Scanner(System.in), System.out);
app.run();
```

Gambar 9: Main.java: membuat Library dan menjalankan LibraryApp

Bagian ini menggabungkan semua konsep sebelumnya menjadi aplikasi konsol. Tanggung jawab dipisah: model (Library, Member, LibraryItem dan turunannya)

mengurus data dan aturan, sedangkan LibraryApp mengurus tampilan dan input. Library menyimpan koleksi di array LibraryItem[] sehingga Book, Dvd, dan Magazine bisa disimpan bersama (polymorphism), serta memakai array Member[] untuk anggota. LibraryApp menerima Scanner dan PrintStream lewat constructor, dan Main adalah satu-satunya titik awal program.

Pikirkan Dulu:

1. Library tidak boleh mencetak apa pun. Mengapa? Apa yang kamu ganti di proyek akhir saat tampilannya menjadi GUI, dan apa yang tidak perlu disentuh?
   Jawab: Agar logika data dan aturan (model) terpisah dari tampilan (separation of concerns). Library hanya mengembalikan data dan true/false, sehingga bisa dipakai oleh tampilan apa pun dan mudah diuji otomatis. Jika tampilan diganti menjadi GUI, yang diganti hanya LibraryApp (kelas tampilan). Library, Member, LibraryItem dan turunannya tidak perlu diubah.
2. Array items berisi Book, Dvd, dan Magazine sekaligus. Mengapa itu diizinkan? Konsep apa namanya?
   Jawab: Karena ketiganya adalah subclass dari LibraryItem, sehingga objek Book, Dvd, atau Magazine boleh disimpan di variabel atau array bertipe LibraryItem ("is-a"). Konsepnya adalah polymorphism (satu tipe, banyak bentuk), dengan penyimpanan objek subclass sebagai tipe induk disebut upcasting.
3. Mengapa LibraryApp menerima Scanner dan PrintStream lewat constructor, bukan langsung memakai System.in dan System.out?
   Jawab: Supaya input dan output bisa diganti. Di program sungguhan kita memberi keyboard dan layar, sedangkan autograder bisa memberi input dari teks dan menangkap output untuk diperiksa. Ini membuat LibraryApp lebih fleksibel dan mudah diuji (dependency injection).
4. Pada alur "pinjam": objek mana yang dipanggil, dan method apa yang berjalan di masing-masingnya?
   Jawab: Jawaban dibawah pada tabel Uji pemahaman

Uji Pemahaman:

1. ikuti satu alur peminjaman dari menu 6 sampai selesai. Sebutkan urutan objek yang dipanggil: LibraryApp, Library, Member, LibraryItem. Method apa saja yang berjalan di setiap objek? Tulis di laporan.
   Jawab:

Urutan	Objek	Method yang dijalankan
1	LibraryApp	run() membaca pilihan "6" dan memanggil lendItem(); readLine("Member ID: ") dan readLine("Title: ") membaca input; lalu memanggil library.lend(memberId, title).

2	Library	lend(memberId, title) memanggil findMember(memberId) dan findItemByTitle(title), lalu memeriksa syarat peminjaman.
3	Member	canBorrow() (apakah loanCount < 3); jika lolos, addLoan() menambah loanCount.
4	LibraryItem (Book/Dvd/ Magazine)	isAvailable() memeriksa status, lalu checkOut() mengubah status menjadi dipinjam dan mengembalikan true.
5	LibraryApp	Menerima hasil true/false dari library.lend lalu mencetak "Loan successful" atau "Loan failed" dengan out.println.

a. Penjelasan kerja sama LibraryApp, Library, Member, dan LibraryItem pada satu alur peminjaman.
Jawab: Kerja sama keempat objek pada satu alur: LibraryApp berperan sebagai perantara pengguna (membaca input dan menampilkan hasil), Library sebagai pengatur yang mencari objek dan menyusun urutan pemeriksaan, Member menjaga aturan batas 3 pinjaman, dan LibraryItem menjaga status tersedia atau dipinjam. Setiap objek hanya mengurus tanggung jawabnya sendiri.

b. Hasil pengujian: screenshot mvn test atau tab Actions yang menampilkan nilai.

```
=== Polinema Library===
1. List items
2. Add book
3. Add DVD
4. Add magazine
5. Register member
6. Lend item
7. Return item
8. Search items
9. List members
0. Exit

Choose: 6
Member ID: M001
Title: Clean Code
Loan successful

=== Polinema Library===
1. List items
2. Add book
3. Add DVD
4. Add magazine
5. Register member
6. Lend item
7. Return item
8. Search items
9. List members
0. Exit
```

Gambar 10: Hasil pengujian menu 6

```
Choose: 1
Clean Code (2008) by Robery Martin | Borrowed | 14 days
Inception (2010) [148 min] | Available | 3 days
Tempo (2024) | Available | 7 days

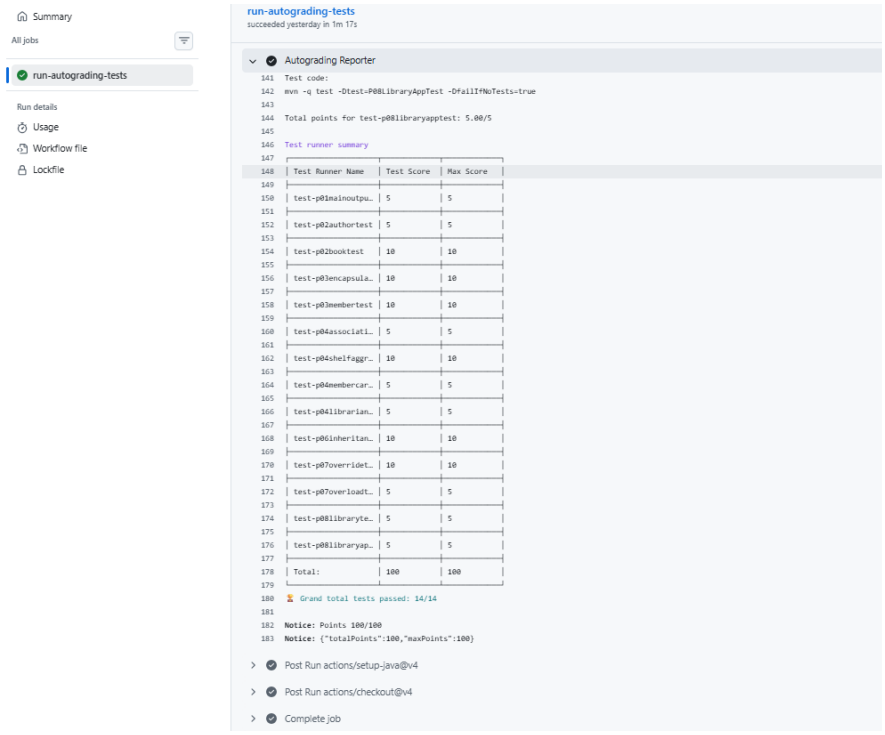
=== Polinema Library===
1. List items
2. Add book
3. Add DVD
4. Add magazine
5. Register member
6. Lend item
7. Return item
8. Search items
9. List members
0. Exit

Choose: 9
M001 - Budi (1 loans)

=== Polinema Library===
1. List items
2. Add book
3. Add DVD
4. Add magazine
5. Register member
6. Lend item
7. Return item
8. Search items
9. List members
0. Exit

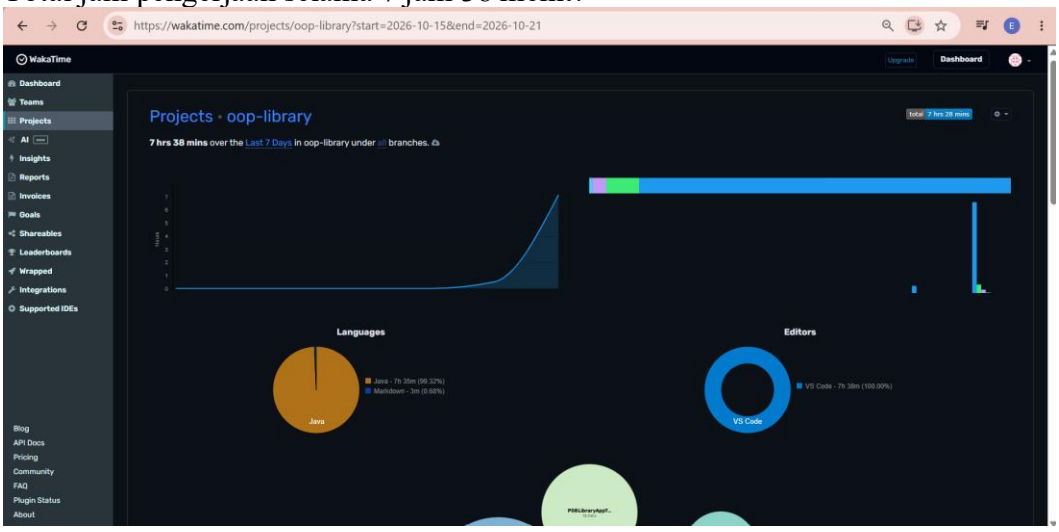
Choose: 0
Goodbye
PS C:\Users\ASUS\OneDrive\Pictures\oop_library-Elsa>
```

Gambar 11: Hasil pengujian menu 1 & 9



Gambar 12: Hasil nilai dari pengujian

c. Bukti WakaTime: screenshot dashboard project oop-library (total waktu dan grafik per hari) dan total jam dalam teks.
Total jam pengerjaan selama 7 jam 38 menit:



Gambar 13: Bukti pengerjaan dengan Wakatime

d. Refleksi: konsep yang masih membingungkan dan rencana belajarmu
Konsep yang paling sulit bagi saya adalah polymorphism pada Bagian 6 dan 7, terutama memahami bahwa variabel bertipe LibraryItem bisa menunjuk objek Book, Dvd, atau Magazine, dan method yang berjalan ditentukan oleh objek aslinya, bukan tipe variabelnya. Awalnya saya mengira loanDays () akan selalu

mengembalikan 7 karena tipe variabelnya Libraryltem. Setelah mencoba array LibraryItem[] dan melihat hasil 14, 3, dan 7, saya mulai paham. Konsep kedua yang masih membingungkan adalah perbedaan agregasi dan komposisi (Shelf dengan Book, Member dengan LibraryCard), karena keduanya sama-sama berupa field dan perbedaannya ada pada siapa yang membuat objek dan apakah objek itu hidup sendiri.

Rencana belajar: (1) mengulang membuat diagram kelas dan menggambar stack dan heap untuk contoh polymorphism dan komposisi; (2) membuat kelas baru kecil dengan hierarki sendiri (misalnya Vehicle, Car, Motorcycle) lengkap dengan override dan overload; (3) membaca ulang materi tentang super, this, dan protected.